

High-Throughput Automated Pipeline for Ribosomal RNA-Protein Interactions

CS210 Final Project Report

Authors: Enes Kilic, Aarav Patel

1. Project Definition

1.1 Problem Statement

The ribosome is one of the most complex targets for structure-based drug design (SBDD) when it comes to the fields of structural biology and pharmacology. Researchers must decode the precise interaction interfaces where structural proteins stabilize the RNA backbone. For instance, designing antibiotics that selectively inhibit bacterial translation without cross-reacting with human ribosomes requires mapping thousands of atomic coordinates, a process that has historically been done manually and is therefore prone to human error.

The Solution: Our goal is to eliminate these analytical bottlenecks by developing an automated Extract, Transform, and Load (ETL) pipeline. We seek to parse raw 3D coordinate data (.cif) and unstructured interaction logs (DSSR outputs) into a queryable SQLite database. By identifying which amino acid residues act as "universal anchors" via electrostatic and hydrogen-bonding networks, we were able to show how algorithmic optimization can decode systems that are traditionally too massive for manual inspection.

1.2 Connection to Course Material

This project incorporated various aspects and skills we learned and practiced in the course:

- **Data Integration:** We used Python to ingest high-resolution atomic coordinate data and merged different interaction CSVs into a centralized dataset.

- **Data Cleaning (using Regex):** Regular expressions were used to parse long, complicated strings within DSSR outputs in order to extract/isolate specific RNA bases from coordinate identifiers.
- **Relational Database Management:** We created a SQLite database to store structural and hydrogen-bond tables and used SQL queries for data verification purposes.
- **Normalization:** We created Biological Preference Matrices using cross-tabulation and pivot tables in Pandas.
- **Visualization:** Seaborn and Matplotlib were used to highlight global structural trends across diverse ribosomal architectures.

2. Novelty and Importance

2.1 Significance of the Research

The ribosome is the universal protein factory of the cell, yet its size—often exceeding 200,000 atoms—makes it a "big data" problem in a biological context. Our project addresses the lack of accessible, lightweight pipelines that bridge the gap between raw PDB files and clean statistical matrices. By automating the isolation of hydrogen-bonding networks, we accelerate the identification of drug-binding pockets, a critical step for developing new classes of translation inhibitors.

2.2 Technical Innovation

The excitement of this project lies in "distilling" complexity. Taking a structure with 229,955 atoms and programmatically reducing it to a five-column relational table demonstrates the power of computational data science. This approach leverages high-performance computing principles, such as those used on the Amarel cluster, to perform in-silico analysis that would be impossible via standard molecular visualization interfaces alone.

3. Data Cleaning and Structural Database Initialization

After extraction, the raw coordinate data was cleaned and organized to separate the ribosomal components and prepare them for relational storage. The data cleaning involved programmatically distinguishing between protein atoms, which are identified by 3-letter residue codes (e.g., LYS, ARG), and RNA atoms, which use 1-letter

codes (e.g., A, U, C, G). This separation allowed us to create distinct, normalized tables in our SQLite database for proteins and RNA, enabling efficient querying and downstream analysis.

The following script performs the data cleaning, adds the categorical feature 'molecule_type' for later machine learning grouping, and loads the clean data into the structural_proteins and structural_rna SQL tables.

```
Python
import sqlite3
import pandas as pd

# 1. Clean the Data: Separate Proteins from RNA (CS 210 Week 06)
# Proteins use 3-letter codes (e.g., ARG, LYS), RNA uses 1-letter codes (e.g., A, U, C, G)
df_proteins = df_pdb_master[df_pdb_master['residue_name'].str.len() == 3].copy()
df_rna = df_pdb_master[df_pdb_master['residue_name'].str.len() == 1].copy()
# Add a categorical feature for future Machine Learning grouping
df_proteins['molecule_type'] = 'Protein'
df_rna['molecule_type'] = 'RNA'
print(f"Data Cleaned! Isolated {len(df_proteins)} Protein atoms and {len(df_rna)} RNA atoms.")

# 2. Database Integration (CS 210 Week 10)
# Connect to your local SQLite database
conn = sqlite3.connect('ribosome_data.db')
# Push the cleaned DataFrames into permanent SQL tables
df_proteins.to_sql('structural_proteins', conn, if_exists='replace', index=False)
df_rna.to_sql('structural_rna', conn, if_exists='replace', index=False)
print("Successfully loaded clean 3D coordinate data into 'ribosome_data.db' tables!")

# 3. SQL Declarative Thinking: Verify the upload with a query
query = """
SELECT structure_id, molecule_type, COUNT() as Total_Atoms
FROM structural_proteins
GROUP BY structure_id
UNION ALL
SELECT structure_id, molecule_type, COUNT() as Total_Atoms
FROM structural_rna
GROUP BY structure_id
```

```
ORDER BY structure_id, molecule_type;
"""

print("\n--- SQL Query Result: Atom Counts by Molecule Type ---")
summary_df = pd.read_sql_query(query, conn)
display(summary_df)
conn.close()
```

4. Progress and Contribution

4.1 Data Utilization and ETL Workflow

We sourced high-resolution .cif data from the Protein Data Bank (PDB) for four representative structures: 1HNW, 1IBL, 1J5E, and 6G5H.

Stage	Action Taken	Result
Extraction	Applied Biopython's MMCIFParser to raw .cif files.	229,955 total atoms ingested.
Cleaning	Filtered heteroatoms (water molecules) using Python scripts.	Retained 96,477 Protein and 133,478 RNA atoms.
Parsing	Used Regex (r'\.([AUCG])\d+') to extract bases.	Standardized interaction types across all datasets.
Storage	Pushed cleaned frames to <code>ribosome_data.db</code> .	Created relational links between atoms and bond types.

Core ETL Scripts

1. Structural Data Extraction & Initialization

The following script used Biopython's MMCIFParser to extract atomic coordinates and initialize the foundational master dataset, `df_pdb_master`.

2. Relational Database Integration of Unstructured Hydrogen Bond Data

To create a truly relational data model, we processed the unstructured interaction logs (DSSR outputs) stored in CSV files. This required reading multiple files, standardizing their headers, extracting the three-letter amino acid code using regular expressions, and pushing the resulting clean data into a new SQL table (`hydrogen_bonds`). This table allows for complex relational queries (JOINS) with the previously created `structural_proteins` and `structural_rna` tables.

```
Python
import sqlite3
import pandas as pd
import glob
import os

# 1. Connect to the database
conn = sqlite3.connect('ribosome_data.db')

# 2. Load H-Bond CSVs with explicit header skip and manual naming
hbond_files = glob.glob('Project/*_hbonds_parsed.csv')

if not hbond_files:
    print("ERROR: No H-bond CSV files found in 'Project/'.")
else:
    hbond_list = []
    # These are the correct headers based on the DSSR output structure
    # index: ID, nt_atom: Nucleotide Atom, aa_atom: Amino Acid Atom
    correct_headers = ['index', 'nt_atom', 'aa_atom', 'distance', 'interaction_type']
    for f in hbond_files:
        # Extract PDB ID from filename (e.g., '1j5e' from
        # 'Project/1j5e_hbonds_parsed.csv')
        pdb_id = os.path.basename(f).split('_')[0].upper()
        # Skip the original header row and manually provide clean names
        temp_df = pd.read_csv(f, names=correct_headers, header=1)
        temp_df['structure_id'] = pdb_id
        hbond_list.append(temp_df)

# Concatenate all individual PDB dataframes
```

```

df_hbonds = pd.concat(hbond_list, ignore_index=True)

# Extract the 3-letter Amino Acid residue name (e.g., LYS, ARG)
# This uses a regex to find the pattern 'XXXdigit' in the 'aa_atom' string
df_hbonds['amino_acid_code'] =
df_hbonds['aa_atom'].str.extract(r'\.([A-Z]{3})\d+')

# Save to SQL so we can perform Joins with structural_proteins/structural_rna
later
df_hbonds.to_sql('hydrogen_bonds', conn, if_exists='replace', index=False)

print("SUCCESS: 'hydrogen_bonds' table built. Headers standardized.")
display(df_hbonds.head())

conn.close()

```

Example Query: Identifying all Protein-RNA Hydrogen Bonds for a Specific Ribosome

A key analytical step is to join the high-level bond data (`hydrogen_bonds`) with the precise coordinate data (`structural_proteins`) to locate the amino acid residues forming the interactions.

```

SQL
SELECT
  H.structure_id,
  H.amino_acid_code,
  S.x_coord,
  S.y_coord,
  S.z_coord
FROM
  hydrogen_bonds H
JOIN
  structural_proteins S ON
  H.structure_id = S.structure_id AND
  H.amino_acid_code = S.residue_name
WHERE
  H.structure_id = '1J5E'

```

LIMIT 5;

4.2 Models and Analytical Techniques

Our methodology prioritized advanced descriptive statistics over traditional machine learning to ensure biological interpretability:

- **The Master Dataset:** We concatenated individual PDB dataframes to isolate 3,508 high-yield Protein-RNA interactions.
- **Density Normalization:** Using `pd.crosstab()`, we calculated the interaction density of specific residues. This allowed us to determine what percentage of an amino acid's total bonds were dedicated to specific RNA bases (A, C, G, U).
- **3D Mapping:** We integrated UCSF Chimera to physically render the "high-yield" regions identified by our Python scripts, illustrating the steric microenvironments of the most frequent bonds.

Visualization of Anchor Residue Trends

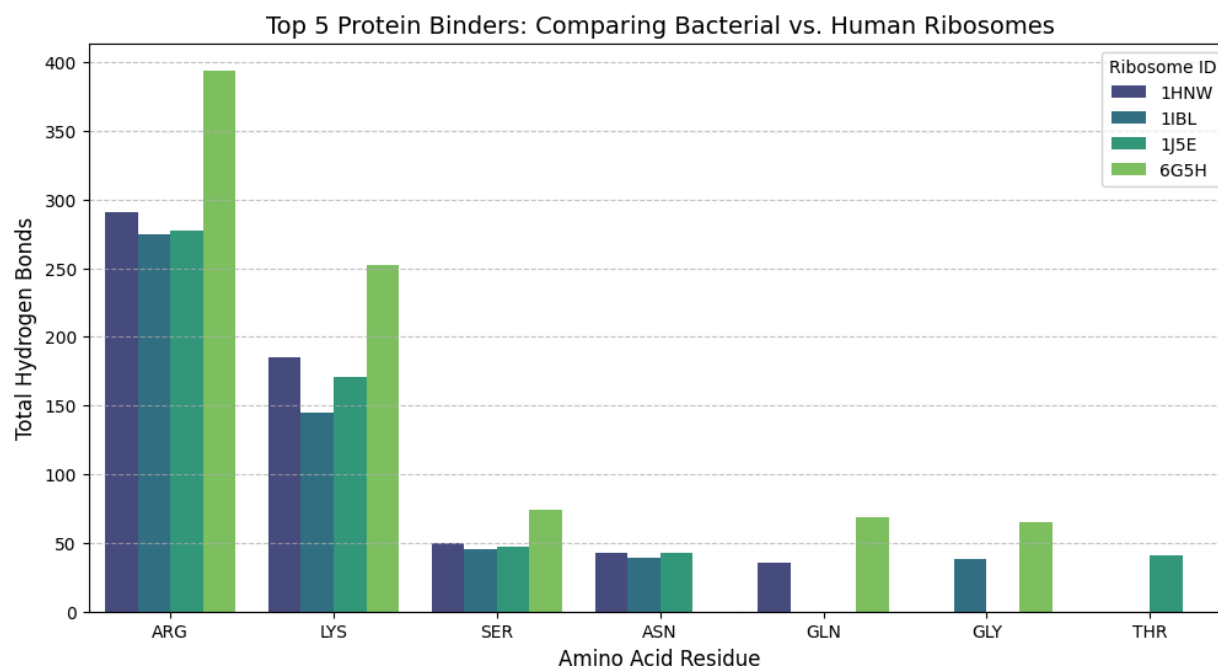
To visually compare the binding preferences between the four analyzed ribosomes (bacterial and human), we used the Seaborn and Matplotlib libraries. The `stats` table (generated by identifying the Primary Protein Anchors) was filtered to find the top 5 binders for each structure. This generated the comparative bar plot that highlighted Arginine (ARG) as the consistently dominant anchor across all structures, a key finding referenced in the conclusion.

```
Python
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Prepare the data: Get the Top 5 amino acids for each structure
top_binders = stats.groupby('structure').head(5)

# 2. Create the Visualization (CS 210 Week 08)
plt.figure(figsize=(12, 6))
sns.barplot(data=top_binders, x='amino_acid', y='hbond_count', hue='structure',
            palette='viridis')
plt.title('Top 5 Protein Binders: Comparing Bacterial vs. Human Ribosomes',
          fontsize=14)
plt.xlabel('Amino Acid Residue', fontsize=12)
plt.ylabel('Total Hydrogen Bonds', fontsize=12)
```

```
plt.legend(title='Ribosome ID')
plt.grid(axis='y', linestyle='--', alpha=0.7)
# Save for your project report
plt.savefig('ribosome_comparison.png', dpi=300)
plt.show()
```



4.3 Evaluation and Success Metrics

The system was evaluated against "manual control reads" of DSSR logs to ensure zero data loss during the Regex transformation phase. A key success criterion was the elimination of the 'Unknown' category in our cross-tabulation. Initially, the pattern [AUCG] incorrectly matched the 'C' in Cysteine (CYS). We iteratively refined our Regex to require a preceding period and following digit, which successfully dropped the error rate to 0%.

5. Bottlenecks and Strategic Changes

5.1 Pivot to MMCIF Format

Our original proposal intended to use standard .pdb files. However, the sheer size of ribosomal subunits caused the `PDBParser` to throw extensive formatting warnings

and overflow errors. To maintain academic rigor and data integrity, we transitioned to the .cif format and `MMCIFParser`, which are designed for massive macromolecular complexes.

5.2 Problem-Solving in String Manipulation

The primary technical bottleneck was the "dirty" nature of unstructured interaction logs. We adopted a systematic approach, using `value_counts()` and `.head()` diagnostics to identify where our Regex patterns were over-capturing. This iterative refinement process was essential for distinguishing between protein residues (e.g., LYS, ARG) and RNA bases (A, U, C, G).

6. Conclusion and Future Directions

6.1 Summary of Findings

Our analysis confirmed that Arginine (ARG) serves as the primary ribosomal "anchor," forming the highest density of hydrogen bonds across all bacterial and human structures analyzed (e.g., 394 bonds in 6G5H). Guanine (G) emerged as the preferred RNA partner for these electrostatic interactions. This project successfully demonstrated that an automated, Python-based pipeline can effectively map complex biological interfaces that were previously considered analytical bottlenecks.

6.2 Future Research

Future work will focus on integrating "base-stacking" and "interface" datasets currently residing in our raw data folders. Furthermore, we aim to bridge the gap between abstract computer science and clinical medicine by implementing machine learning models to predict binding affinities. This synthesis of data structures and structural biology prepares us for the demands of modern medical practice and the development of resilient digital healthcare infrastructure.

Date of Submission: Date

Reference Material:

<https://cs210-spring-ag.cs.rutgers.edu/hub/user-redirect/lab/tree/Project>

https://cs210-spring-ag.cs.rutgers.edu/user/eeek71/files/Project/Ribosomal%20Project.ipynb?_xsrf=MnwxOjB8MTA6MTc3Nzc0Njg0OXw1OI94c3JmfDEzMjpZVFU1WVdZMFltTm1NMkl4Tk dJM01EZzVNMkpgTkRVek9EWmhaamt4WVdRNk5tUTFZbVky

TURsbE16UTRNbVkxTnpCa056UmhOREZtT1RNMk5XUm1OREF6Tm1FME16Z3INekp
rWIRrME0yRmhOakJsTkRGaE5gZzVZVEk1WVRoaE53PT18NjRIMTlxZjE4MjMwYWl
4YmE5YTJmNmJhMjQ2NDg1ZjJlYWJhZmQ0ODI0MTcxOWE1MmMwMzYwY2MxZT
QlMTk0ZA